

Assignment 3

2IMP20 - DSL Design

June 3, 2026

1 Introduction

In this assignment, you have to design and implement your own Domain-Specific Language (DSL). You are free to design and implement your own executable language, but it should be non-trivial and include concepts that have behaviour. Thus, markup languages (e.g., HTML and XML) are not allowed; neither are languages with only a few language constructs, like IFTTT. **Your language has to be such that processing (transformation to a GPL, interpretation) involves sufficient *behaviour***, i.e. it should not be mostly a data format language such as JSON, or one for which transformation or interpretation does not involve substantial effort. The basic LaBouR language of Assignments 1 and 2 would be too simplistic as well.

You may re-implement an existing language as long as you refer to the source where you found this language. You could also choose to implement an existing state machine language or some toy robot language. There are sufficient examples of DSLs you can choose from or get inspired by. With permission from the students involved, we have also made available three projects including their reports from a previous year. These serve as inspiration and indication of what (very) good, though not necessarily perfect, projects and reports look like. You can find them as the files named Example-... at <https://canvas.tue.nl/courses/32874/files/folder/assignments>.

2 Assignment

The third assignment is meant to apply the concepts of the course to implement a DSL from scratch. The goal of the assignment is to build an executable DSL of your choice. You have to specify the syntax for the language; this involves concrete and abstract syntax. In addition, you have to define the static semantics of the language and its implementation. It is advisable to start with simple static semantic rules and gradually extend them. Finally, you must define a mapping from your DSL to a language that can be executed, for instance, C, C++, Java, or Python. This mapping can be implemented in different ways; choose a straightforward solution, for example, using templates (code generation).

You are free in the choice of your language workbench. In the second assignment, you used Rascal, but if you want to explore other language workbenches, such as EMF with Xtext and ETL/QVTo, MPS, or Spoofax, you can do this. However, be aware that other language workbenches have their own (steep) learning curve, take effort to get installed, and may lack proper documentation.

As part of the assignment, **each of you** is required to select and read two scientific papers related to the design/implementation of DSLs. Take the set of 2+2 papers you got approval for as a group, and partition these amongst members of your group, such that each member studies 1 general and 1 domain-focused paper, with no overlap between papers studied by the group members. These papers should provide context and better insight into the work you performed, and help you to reflect on it.

While the DSL design and implementation part of this assignment must be done in pairs, you are required to submit an individual report containing reflection on your experiences as a language designer, and discussion of at least the two scientific papers allocated to you. It is advisable to select and read your papers early on, and to discuss them with your group partner to improve understanding.

3 Deliverable

For the third assignment, you must hand in the following deliverable:

- Individual report (expectation: ca. 4 pages) containing:
 1. Discussion of the 2 scientific papers you have selected and read related to the design/implementation of DSLs. These papers should provide you better insight into the work you performed. You should *briefly* summarize each paper, but more importantly critically reflect on it and relate it to your work i.e. to your DSL and/or the process of developing your DSL.
 2. A reflection on your experiences as a language designer. You must (a) reflect on the use of the chosen LWB and your DSL's design and implementation, including motivation of (design) decisions; and (b) reflect what you learned by taking this course and doing the assignments, including by designing and implementing a DSL in this assignment.
 3. A reflection on the guest lectures, both on their topics and on the connection and relevance to your project, or why there is none.
- *You must work together with your partner to create the artifacts and documentation mentioned below.*
- All artifacts you have created, syntax definitions, static semantic rules and implementation, and code generators.
- Several non-trivial input programs or models and the corresponding generated outputs.
- Technical description/documentation of your language in the form of a report (expectation: ca. 10 pages), including references to the sources you have used as inspiration, and a motivation for choosing this language, including the purpose of this language.
- Video of between 5–10 minutes where you show the language definition (both syntax and semantics), the creation of a program/model for this language, the transformation to source code, and the compilation and execution of the generated code. *NB: longer or shorter videos will lead to 0 points for the video.*

4 Grading

The grading will be based on the following aspects:

- Reflection (4pt in total), covering
 - Individual discussion and reflection on the 2 scientific papers (1 pt);
 - Your experiences as a language designer, including (a) the use of the chosen LWB and your DSL's design and implementation, including motivation of (design) decisions (1 pt); and (b) what you learned by taking this course and doing the three assignments (0.5 pt);
 - The guest lectures, both on their topics and on the connection and relevance to your project, or why there is none (up to 1.5 point if you attended 2+ guest lectures; up to 0.5 point if you attended exactly 1 guest lecture; 0 points otherwise).
- Description/documentation of the language (1 pt)
- Abstract and concrete syntax description (1 pt)
- Description of static semantic rules (1 pt)
- Implementation of the language (1 pt)
- Non-trivial input files and generated output (1 pt)
- Quality of the created video (1 pt)

5 Submission

You have to submit a single zip file containing all created files and the test programs, generated output documentation, video *and the individual report for each group member*.

You have to submit this zip file *as a Canvas group of two students* by **23:59 on Friday the 3rd of July** via Canvas.